2.4.1 A short detour: writing loops functionally

First, let's write `factorial`:

```
def factorial(n: Int): Int = {
  def go(n: Int, acc: Int): Int =
    if (n <= 0) acc
    else go(n-1, n*acc)

  go(n, 1)
}
```

An inner function, or *local definition*. It's common in Scala to write functions that are local to the body of another function. In functional programming, we shouldn't consider this a bigger deal than local integers or strings.

The way we write loops functionally, without mutating a loop variable, is with a recursive function. Here we're defining a recursive helper function inside the body of the `factorial` function. Such a helper function is often called `go` or `loop` by convention. In Scala, we can define functions inside any block, including within another function definition. Since it's local, the `go` function can only be referred to from within the body of the `factorial` function, just like a local variable would. The definition of `factorial` finally just consists of a call to `go` with the initial conditions for the loop.

The arguments to `go` are the state for the loop. In this case, they're the remaining value `n`, and the current accumulated factorial `acc`. To advance to the next iteration, we simply call `go` recursively with the new loop state (here, `go(n-1, n*acc)`), and to exit from the loop, we return a value without a recursive call (here, we return `acc` in the case that `n <= 0`). Scala detects this sort of *self-recursion* and compiles it to the same sort of bytecode as would be emitted for a `while` loop,[3] so long as the recursive call is in *tail position*. See the sidebar for the technical details on this, but the basic idea is that this optimization[4] (called *tail call elimination*) is applied when there's no additional work left to do after the recursive call returns.

Tail calls in Scala

A call is said to be in *tail position* if the caller does nothing other than return the value of the recursive call. For example, the recursive call to `go(n-1,n*acc)` we discussed earlier is in tail position, since the method returns the value of this recursive call directly and does nothing else with it. On the other hand, if we said `1 + go(n-1,n*acc)`, `go` would no longer be in tail position, since the method would still have work to do when `go` returned its result (namely, adding `1` to it).

If all recursive calls made by a function are in tail position, Scala automatically compiles the recursion to iterative loops that don't consume call stack frames for each iteration. By default, Scala doesn't tell us if tail call elimination was successful, but if we're expecting this to occur for a recursive function we write, we can tell the Scala

[3] We can write `while` loops by hand in Scala, but it's rarely necessary and considered bad form since it hinders good compositional style.

[4] The term *optimization* is not really appropriate here. An optimization usually connotes some nonessential performance improvement, but when we use tail calls to write loops, we generally rely on their being compiled as iterative loops that don't consume a call stack frame for each iteration (which would result in a `StackOverflowError` for large inputs).